

Practica 01: El juego de caos

El juego del caos consiste simplemente en ir dibujando puntos de forma aleatoria, de tal forma que dicha sucesión de puntos en el límite nos va a dar una fractal conocido.

Consideremos un triángulo con vértices A, B, C (Comúnmente es utilizado en triángulo equilátero) y ejecutemos el siguiente algoritmo:

1. Elige un punto x_0 en el plano.
2. Elige aleatoriamente uno de vértices A, B, C .
3. El punto siguiente x_{i+1} será el punto medio entre el punto x_i y el vértice elegido.
4. Volver al paso 2.

Programa una función que realice el algoritmo anterior y ejucatalo 10 000 veces.

Hint: Para escoja aleatoriamente entre los tres vertices, puedes utilizar `np.random.uniform()`.

Enlace para descargar Anaconda: [Presióname!!](#).

Enlace de colab: [Pulsa aqui :3.](#)

#Escribe aqui tu código

Grafica

Si tienes problemas para graficar, puedes ver varios ejemplos en la siguiente pagina, [Haz click aqui >u<](#).

Introducción a Python

Es un lenguaje interpretado, dinámico y multiplataforma. En la última década, ha estado en las listas de los lenguajes de programación más utilizados. Una de las razones es su utilización en cualquier área, como la creación de sitios web, Inteligencia Artificial, video juegos, microsisistemas, Big Data, etc.

Tipos de datos

Una variable puede contener valores de diversos tipos.

String o cadenas de texto

```
cadena = 'Hola mundo'
print(cadena)
```

Hola mundo

```
cadena = '''Esta es una  
cadena de varias  
líneas'''
```

```
print(cadena)
```

Esta es una
cadena de varias
líneas

#Concatenar

```
mensaje = 'Hola' + ' ' + 'Mundo'  
print(mensaje)
```

#Contar el largo de la cadena

```
print(len(mensaje))
```

#Encontrar una cadena, indicará el índice de inicio de la misma.

```
print(mensaje.find('Mundo'))
```

#multiplicar

```
mensaje2a = 'Hola ' * 3  
mensaje2b = 'Mundo'  
print(mensaje2a + mensaje2b)
```

Numéricos

#números enteros

```
entero = 23
```

#números reales

```
real = 37.45
```

Booleanos

```
verdadero = True
```

```
falso = False
```

Tuplas

Una **tupla** es una variable que permite almacenar varios datos inmutables (**no pueden ser modificados una vez creados**) de tipos diferentes.

```
mi_tupla = (True, 15.3, 4, 'cadena')
```

#Se puede acceder a cada uno de los datos mediante su índice correspondiente

#Recordemos que python comienza desde 0

```
print(mi_tupla[0])
```

Otra forma de acceder a la tupla de forma inversa (de atrás hacia

adelante), es colocando un índice negativo

```
print(mi_tupla[-2])
```

0 a una porción de ellos

```
print(mi_tupla[1:], mi_tupla[:3], mi_tupla[1:2])
```

```
True
```

```
4
```

```
(15.3, 4, 'cadena') (True, 15.3, 4) (15.3,)
```

#Intentemos modificar un elemento de una tupla

```
mi_tupla[2] = 45
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
<ipython-input-7-ala261ef4646> in <module>
```

```
    1 #Intentemos modificar un elemento de una tupla
```

```
----> 2 mi_tupla[2] = 45
```

```
TypeError: 'tuple' object does not support item assignment
```

Listas

Una **lista** es similar a una tupla con la diferencia fundamental de que permite modificar los datos una vez creados.

```
mi_lista = ['cadena', 2.8, 'otro dato', False, 15]
```

#Se accede de la misma forma que las tuplas, por índice

```
print(mi_lista[3])
```

```
print(mi_lista[1:3])
```

```
print(mi_lista[-2])
```

#Longitud de lista

```
print(len(mi_lista))
```

```
False
```

```
[2.8, 'otro dato']
```

```
False
```

```
5
```

Las lista **NO son inmutables**: permiten modificar los datos una vez creados.

```
mi_lista[2] = (1, 2, 3)
```

```
print(mi_lista)
```

```
['cadena', 2.8, (1, 2, 3), False, 15]
```

Métodos de una lista

.append() - Este método agrega un elemento al final de una lista.

```
mi_lista.append('Soy el último elemento')
print(mi_lista)
```

```
['cadena', 2.8, (1, 2, 3), False, 15, 'Soy el último elemento']
```

.count() - Este método recibe un elemento como argumento, y cuenta la cantidad de veces que aparece en la lista.

```
print(mi_lista.count(15))
```

```
1
```

.insert() - Este método inserta el elemento x en la lista, en el índice i.

```
mi_lista.insert(1, 'FC')
print(mi_lista)
```

```
['cadena', 'FC', 2.8, (1, 2, 3), False, 15, 'Soy el último elemento']
```

.pop() - Este método devuelve el último elemento de la lista, y lo borra de la misma.

```
mi_lista.pop()
print(mi_lista)
```

```
['cadena', 'FC', 2.8, (1, 2, 3), False, 15]
```

.sort() - Este método ordena los elementos de una lista. Tiene por defecto reverse = False ordena del más chico al más grande.

```
numeros = [17, 6.7, 4.8, 9, 12]
numeros.sort(reverse = True)
print(numeros)
```

```
[17, 12, 9, 6.7, 4.8]
```

Diccionarios

Mientras que a las listas y tuplas se accede solo y únicamente por un número de índice, los diccionarios permiten utilizar una clave para declarar y acceder a un valor

```
mi_diccionario = {'clave_1': 13, 'clave_2': True, 'clave_3': 'Valor3',
                  'clave_4': [1, 2, 3]}
print(mi_diccionario['clave_4'])
```

```
[1, 2, 3]
```

Un diccionario permite **eliminar** cualquier entrada.

```
del(mi_diccionario['clave_2'])
print(mi_diccionario)
```

Al igual que las listas, el diccionario permite **modificar** los valores.

```
mi_diccionario['clave_1'] = 'Nuevo Valor'
print(mi_diccionario)

{'clave_1': 'Nuevo Valor', 'clave_2': True, 'clave_3': 'Valor3',
'clave_4': [1, 2, 3]}
```

Métodos de un diccionario

.keys() - Genera una lista con las claves del diccionario.

```
mi_diccionario.keys()

dict_keys(['clave_1', 'clave_2', 'clave_3', 'clave_4'])
```

.values() - Genera una lista con el valor de los registros del diccionario.

```
mi_diccionario.values()

dict_values(['Nuevo Valor', True, 'Valor3', [1, 2, 3]])
```

.clear() - Borra todos los registros de un diccionario

```
mi_diccionario.clear()
print(mi_diccionario)

{}
```

.pop() - Extrae un registro de un diccionario a partir de su clave y lo borra, acepta valor por defecto.

```
mi_diccionario = {'clave_1': 13, 'clave_2': True, 'clave_3': 'Valor3',
'clave_4': [1, 2, 3]}
mi_diccionario.pop("oveja", "no se ha encontrado")
#mi_diccionario.pop('clave_1', 'upps')

'no se ha encontrado'
```

.get() - Busca un elemento a partir de su clave y si no lo encuentra acepta un valor por defecto.

```
mi_diccionario.get('oveja','no se encuentra')

'no se encuentra'
```

Asignación múltiple

```
a, b, c = 'string', 15, True
print(b)
```

```
mi_lista = ['México', 'San Luis Potosi']
país, estado = mi_lista
print(país)
```

México

```
mi_tupla = ('Kora', 15000.00 )
ID, dinero = mi_tupla
print(ID)
```

Estructuras de Control de Flujo

En Python, la indentación (4 espacio en blanco) es obligatoria, ya que de ella, dependerá su estructura.

Las estructuras de control condicionales, son aquellas que nos permiten evaluar si una o más condiciones se cumplen, para decir qué acción vamos a ejecutar. La evaluación de condiciones.

Operadores de comparación

```
print(1 == 2) # Igual que
print(1 != 2) # Distinto que
print(1 < 2) # Menor que
print(1 > 2) # Mayor que
print(1 <= 2) #Menor o igual que
print(1 >= 2) #Mayor o igual que
```

```
False
True
True
False
True
False
```

Operadores lógicos

```
print((1 == 1) and (1 < 2)) # 0 y 0 = False, 1 y 1 = True, 1 y 0 =
False, 0 y 1 = False.
print((1 == 1) or (1 > 2)) # 0 y 0 = False, 1 y 1 = True, 1 y 0 =
True, 0 y 1 = True.
```

```
True
True
```

if, elif, else

```
semaforo = 'amarillo'
```

```
if semaforo == 'verde':
    print("Cruzar la calle")
elif semaforo == 'amarillo':
    print('Disminuir velocidad')
```

```
else:  
    print("Esperar")
```

Disminuir velocidad

Ciclos while

```
cuenta = 0  
while cuenta <= 200:  
    print(cuenta)  
    if cuenta == 5:  
        break  
    cuenta = cuenta + 1
```

0
1
2
3
4
5

Ciclos for

```
mi_lista = ['Juan', 'Antonio', 'Pedro', 'Herminio']  
for nombre in mi_lista:  
    print(nombre)
```

Juan
Antonio
Pedro
Herminio

```
mi_tupla = ('rosa', 'verde', 'celeste', 'amarillo')  
for color in mi_tupla:  
    print(color)
```

rosa
verde
celeste
amarillo

```
for i in range(1, 10):  
    print('*u*')  
    if i == 5:  
        break
```

u
u
u
u
u

Funciones

La definición de funciones se realiza mediante la instrucción **def** más un nombre de función descriptivo.

#Un parámetro es un valor que la función espera recibir cuando sea llamada (invocada), a fin de ejecutar acciones en base al mismo
#En Python, también es posible, asignar valores por defecto a los parámetros de las funciones.

```
def multiplica(a, b, imprimir = False):  
    x = a*b  
    if imprimir == True:  
        print(x)  
    return(x)
```

#Una función, no es ejecutada hasta tanto no sea invocada.

```
multiplica(5, 5)  
print('****')  
multiplica(5, 5, False)
```

25

Numpy

Numpy es la principal librería para la computación científica en Python. Proporciona un objeto de matriz multidimensional de alto rendimiento y herramientas para trabajar con estas matrices.

#importar la librería

```
import numpy as np
```

```
a = np.array([1, 2, 3])    # Creamos un arreglo de dimensión 1
```

```
print(a)
```

```
print(type(a))            # dato tipo: 'numpy.ndarray'
```

```
[1 2 3]
```

```
<class 'numpy.ndarray'>
```

```
print(a.shape)
```

dimension de arreglo

```
print(a[0], a[1], a[2])   # Prints "1 2 3"
```

```
(3,)
```

```
1 2 3
```

Funciones matematicas

```
x = np.array([1, 2, 3])
```

```
a = np.sqrt(x) #raiz cuadrada
```



```

print('La raíz de' + str(x) + ' es:', a)

b = np.exp(x) #exponencial
print('La exponencial de' + str(x) + ' es:', b)

c = np.log(x) #logaritmo
print('El logaritmo de' + str(x) + ' es:', c)

d = np.cos(x) #coseno
print('El coseno de' + str(x) + ' es:', d)

e = np.sin(x) #seno
print('El seno de' + str(x) + ' es:', e)

La raíz de[1 2 3] es: [1.          1.41421356  1.73205081]
La exponencial de[1 2 3] es: [ 2.71828183  7.3890561  20.08553692]
El logaritmo de[1 2 3] es: [0.          0.69314718  1.09861229]
El coseno de[1 2 3] es: [ 0.54030231 -0.41614684 -0.9899925 ]
El seno de[1 2 3] es: [0.84147098  0.90929743  0.14112001]

```

```
np.pi
```

```
3.141592653589793
```

```
np.exp(1)
```

```
2.718281828459045
```

np.zeros((a, b)) - Crea un array de 0 compuesto de “axb” elementos.

np.ones((a, b)) – Crea un array de 1 compuesto de axb elementos.

```
ceros = np.zeros((3,4)) # Create an array of all zeros
print(ceros)
```

```
print('*****')
```

```
unos = np.ones((2, 4))
print(unos)
```

```

[[0.  0.  0.  0.]
 [0.  0.  0.  0.]
 [0.  0.  0.  0.]]
*****
[[1.  1.  1.  1.]
 [1.  1.  1.  1.]]

```

np.linspace(start,stop,num,endpoint=True) – Crea un array con valor inicial start, valor final stop y num elementos.

```
np.linspace(0, 5, 25)
```

```
array([0.          , 0.20833333, 0.41666667, 0.625          , 0.83333333,
       1.04166667, 1.25          , 1.45833333, 1.66666667, 1.875          ,
       2.08333333, 2.29166667, 2.5          , 2.70833333, 2.91666667,
       3.125          , 3.33333333, 3.54166667, 3.75          , 3.95833333,
       4.16666667, 4.375          , 4.58333333, 4.79166667, 5.          ])
```

np.dot(a, b) - Multiplica las matrices a y b.

```
a = np.array([[8, 2], [8, 4]])
b = np.array([[3, 2], [1, 4]])
c = np.dot(a,b)
print(c)
```

```
[[26 24]
 [28 32]]
```

a*b *#Multiplicación entrada por entrada*

```
array([[24, 4],
       [ 8, 16]])
```

```
a = np.array([[8, 2], [8, 4]])
b = np.array([[3, 2], [1, 4]])
d=np.multiply(a, b) ##Multiplicación entrada por entrada
print(d)
```

```
[[24  4]
 [ 8 16]]
```

.T - obtiene la matriz transpuesta

d.T

```
array([[24,  8],
       [ 4, 16]])
```

numpy.random.uniform(low, high, size) - Una muestra (de tamaño *size*) aleatoria con distribución uniforme en el intervalo *low* a *high*.

```
np.random.uniform(low=0.0, high=1.0, size=None)
```

```
0.8214810380456895
```

```
np.random.uniform(low=0.0, high=1.0, size= 10)
```

```
array([0.72334334, 0.13437586, 0.41083885, 0.57736846, 0.15505458,
       0.52015125, 0.9643721 , 0.05466667, 0.62012205, 0.88044187])
```

Resultado esperado del ejercicio.

Sorprendentemente el resultado es uno de los fractales más conocidos: ¡el triángulo de Sierpinski!